BATTLESHIP!!!

- Riddhi Sanghvi (960153)

Introduction:

The Battleship game is designed to be played on a single computer. The underlying implementation of the Battleship game is Python, and this document details how the architecture of the Battleship Game and how it is tested. The game follows the usual rules of Battleship as well as certain enhancements such as hidden ship placement, alternating (or turn-based) playing on one computer, auto win detection, and long-term recording of high scores on a JSON file.

System Design Overview:

This application follows an object-oriented design approach. There are 5 major components:

1. ShipType
2. Ship
3. Board
4. Game
5. Player

The modular approach of breaking down game logic into separate classes provides the ability to modularize, maintain, and test game logic easily without impacting other aspects of the game.

Class Design Description

Class: ShipType

This class defines the different categories of the ship. This is accomplished by allowing each ship type to be defined in a structured, yet reusable way using this class.

Attribute:

- name (str) - Name of the Ship (Destroyer, Battleship, Cruiser, Aircraft Carrier)
- length (int) - Number of spaces that the ship occupies
- count (int) - Number of ships in each category

Class: Ship

Class Ship models a ship that lays on a game board, tracking its position, its hits and whether or not it has been destroyed.

Attributes:

- ship_type(Type): This is the type of the ship.

- length(int): This is the number of cells the ship will occupy.
- orientation(str): Orientation of placement. It will be either H (horizontal) or V (vertical).
- mid (Tuple[int, int]): Coordinates of the mid-point of the ship for its placement.
- cells (List of Tuple[int,int]): All registered coordinates of the ship that are located on the board.
- hits (set of Tuple[int,int]): Cells that the ship has received hits on.

Methods:

- compute_cells() -> List[Tuple[int,int]]: This method calculates the coordinates of all the occupied cells based on the coordinates of the mid-point of the ship and the orientation.
- is_at (r,c)->bool: The is method checks whether the given cell coordinates is occupied by the ship.
- register_hit (self, r, c): This method marks a cell as hit if the cell coordinates are occupied by the ship.
- is_sunk () -> bool: Returns true if there have been hits on all the cells occupied by the ship.
- remaining_hits() -> int: Returns the number of remaining hits that will be required to destroy the ship.

Class: Board

Represents a board for players and manages ships being placed and removed, and for checking ship hits on the board.

Attributes:

- Ships (List[Ship]): All ships on the Board will be stored in the ships: list.
- occupants(Dict[Tuple[int,int], Ship]): a dictionary containing every (x,y) coordinate of the Board with the ship that is currently occupying the position, providing a quick reference for the ship.

Methods:

- can_place_ship(ship_type, mid_r, mid_c, orientation) -> Tuple[bool, Optional[str]] - returns true or false with an optional error message stating why the ship cannot be placed.
- place_ship(ship_type, mid_r, mid_c, orientation) -> Ship: returns the placed ship with information that allows you to track the ship's location on the board.
- remove_all_ships(): removes all ships from the board.
- remove_ship(ship:Ship): will remove a specifically selected ship.
- all_sunk()->bool: Checks to see if there are any remaining ships on the board.

- ship_at(r,c) ->Optional[Ship]: checks the specified (x,y) coordinate of the board to see which ship is currently occupying the coordinate, or returns none.

Class: Player
Definition: The Class Player houses the characteristics of a player in the game and tracks their respective boards and guessed coordinates.

Attributes:
- Name (str): which stores the name of the player.
- board (Board): which is a Board instance for the player and contains the ships of the player.
- guesses(Dict[Tuple[int,int], str]): Dictionary: a dictionary mapping the coordinates guessed by the player, marked as 'H' (hit), 'M' (miss).
- guess_count (int): which stores the total number of guesses made by a player.

Methods:
- record_guess(r: int, c: int, result): which are used to record the result of a guess and increment the player's guess count.
- has_guessed(r: int, c: int): this method checks to see if a player has previously made a guess at the coordinate (i.e. guessed or not).

Class: Game
Definition: The Class Game governs the state of both players in the game by tracking turns, guesses, and conditions for a winner.

Attributes:
- players (List[Player]): which is made up of the two players in the game.
- active_index (int): which stores the index of the player who is currently playing (either 0 or 1).
- total_turns (int): which is the total number of turns played during this game.

Methods:
- switch_turn(): which switches the active player and increments the turn counter.
- make_guess(r: int, c: int): this method allows the active player to guess a coordinate, providing a response (MISS, HIT, SHIP SUNK) plus the name of the sunk ship.
- is_over(): this method indicates whether the game has concluded (i.e. one player has lost all their ships).
- winner(): returns the winner of the game or None if the game is still ongoing.

Storing the records:

The game results are stored in a JSON file called battleship_records.json
This file contains the following details:

- Timestamp
- Winner's name
- Winner's number of guesses
- Remaining ships
- Total turns
- Player's names
  This file helps with keeping a track of the high scores played to date.

Game Flow:

1. Display a welcome message and retrieve the high score from the previous records, and display it.
2. Players enter their names
3. Initialize the empty board for both players.
4. Explain the board layout (26 rows (A-Z) and 10 columns (0-9)) and that the ship needs to be placed in the middle cell for player 1
5. For each ship type (destroyer, Battleship, Cruiser, Aircraft Carrier), prompt player 1 to enter the middle cell and the orientation of the ship.
6. Validate the placement of the ship so that it does not go out of the board or overlap with any other placed ships. If any of the conditions are not satisfied, then prompt player 1 to enter the values again.
7. After all the ships are placed for player 1, display their board and ask if they want to confirm their placement, replace any of the particular ships, or they want to replace all the ships.
   a. If the player confirms the placement, then go to step 10
   b. If the player wants to replace a particular ship, then display the current placement of all the ships and ask which ship the player wants to replace. After that, accept the cell coordinates of the ship and the orientation. Also, the input for the ship type they are replacing. And replace the ship
   c. If the player wants to replace all the ships, then clear the current placements of the ships on the board and perform steps 4-6.
8. Prompt player 1 to hand over the keyboard to player 2
9. Repeat steps 4-8 for player 2.
10. Ask if both players are ready to start the game
11. Set Player 1 as the active player
12. Begin the turn-based guessing:

a. Display the active player's guess board (. for unknown, 0 for misses and X for Hits)
b. Prompt the active player to enter the row and column for the position they want to guess (e.g., A,1)
c. Validate the input format and ensure the coordinates have not been guessed before.
d. Check the other players' boards at the guessed coordinates:
    i. If no ship is present, then record it as a MISS (0) and display MISS
    ii. If a ship is hit, then record it as a HIT (X) and display (HIT).
    iii. If the hit sinks a ship, i.e., all the cells of the ship are hit, then display SHIP SUNK along with the ship type
e. Increment the active player's guess count
f. Check if all the ships of a player are sunk. If yes, then the game ends; go to step 13.
g. If not, then the turn switches, and the other player goes through the same steps after being prompted to hand over the keyboard to the other player.

13. If the game ends, declare the winner of the game with the player remaining ships.
14. Display both the players' final boards:
    a. Show the misses (0), hits (X), and unhit ships (S).
15. Record the data in the JSON file
16. Show the message that says that the records are saved.

Testing

Implemented 4 classes for testing:

1. test_board.py: These tests confirm the proper functioning of the ship placement and board verification features.
    - can_place_ship(): Confirms that ships that are placed within the board are deemed as placed.
    - test_out_of_bound_reject(): test to check if the ship is placed outside of the board, and if it is rejected.
    - test_ship_collision_reject(): Confirms that two ships cannot occupy the same space on the board.
    - test_ship_lookup(): Confirms that at a given index on the board, the ship at that particular index is revealed.
    - test_top_horizontal_out_of_bound(): Confirm that placing the ship at the top, i.e., A,0, with orientation H, should fail.
    - test_top_vertical_out_of_bound(): Confirm that placing the ship at the top, i.e., A,0, with orientation V, should fail.

2. test_ship.py: Ship behaviours are tested by the following:
    - test_horizontal_ship_cell_generation(): Confirms that the correct cells are generated when a ship is placed horizontally.
    - test_vertical_ship_cell_generation(): Confirms that the correct cells are generated when a ship is placed vertically.
    - test_ship_hit_and_sunk(): Confirms that, once all cells of a ship are hit, that vessel will be marked as sunk.
3. test_game.py: The core logic of the game is validated by the following:
    - test_game_hit(): Confirms that, once a targeted vessel has been hit, it will be confirmed.
    - test_game_ship_sunk(): Confirms that if all cells of a vessel have been hit, it will be marked as a sunk vessel.
    - test_turn_switching(): Confirms that the active player is switched each time the turn ends.
4. test_system.py: The full game flow from start to finish is validated by:
    - test_complete_game(): A simulation of a complete game that demonstrates that one player has won by completely sinking his/her opponent's ship.

Conclusion

The implementation of a two-player Battleship game in Python is a complete success. The implementation follows all the rules of playing the game, such as secretly placing ships, guessing on alternate turns, detecting hits and misses, sinking ships, determining a winner, and so forth. The project is organized using an object-oriented design with clear separation of responsibilities between four classes: Ship, Board, Player, and Game. Each component of the project has been fully tested using both unit and system testing; therefore, we know that the code works as intended throughout the entire lifecycle of playing the game. Additionally, file-based record keeping allows players to retain information about their previous games and the highest score they earned from playing. In summary, this project is an excellent representation of using the Python programming language to implement a game, create logic for games, and perform software testing.